HANGMAN GAME

CodeAlpha Python Programming Internship - Task 1

Project Name:	Hangman Game
Author:	Abdul Samad
Organization:	CodeAlpha
Program:	Python Programming Internship
Date:	May 31, 2026
Task Type:	Task 1 (Compulsory)
Status:	Completed

1. PROJECT OVERVIEW

The Hangman Game is a classic text-based game implemented in Python where a player attempts to guess a hidden word by suggesting letters within a limited number of incorrect guesses. This project demonstrates fundamental Python concepts including loops, conditionals, strings, lists, and functions. The game randomly selects a word and allows the player 6 incorrect attempts before losing.

Game Objective:

• Guess the randomly selected word letter by letter
• Player has a maximum of 6 incorrect guesses
• Win by revealing all letters before running out of guesses
• Lose if 6 incorrect guesses are made

2. REQUIREMENTS & SPECIFICATIONS

Requirement	Status
Create text-based Hangman game	COMPLETED
Use 5 predefined words	COMPLETED
Limit incorrect guesses to 6	COMPLETED
Basic console input/output	COMPLETED
No graphics or audio	COMPLETED
Key concepts: random, loops, if-else, strings, lists	COMPLETED

3. FEATURES IMPLEMENTED

Core Features:
• Random word selection from 5 predefined words
• Letter guessing mechanism with real-time validation
• Hangman ASCII art visualization (7 progressive stages)
• Game state display with progress tracking
• Win/Loss conditions with automatic detection
• Multiple rounds support for continuous play

User Experience Enhancements:
• Clear formatting with visual separators
• Feedback messages for correct/incorrect guesses
• Duplicate guess prevention
• Sorted display of guessed letters
• Game statistics (total guesses, incorrect attempts)
• Friendly prompts and instructions

4. TECHNICAL IMPLEMENTATION

Word List & Selection:
The game uses 5 predefined programming-related words: Python, Hangman, Computer, Programming, and Internship. A word is randomly selected at the start of each game using Python's random module.

Game State Management:
The game maintains key state variables including the current word, guessed letters, display representation with blanks and revealed letters, and remaining attempts. This state is updated with each player action.

Core Functions:
• **initialize_game():** Sets up initial game state
• **display_hangman(tries):** Shows ASCII art based on remaining tries
• **display_game_state():** Shows current progress and statistics
• **get_valid_guess():** Validates and processes player input
• **process_guess():** Updates game state based on guess
• **play_hangman_game():** Main game loop orchestration
• **main():** Enables multiple game rounds

5. TESTING & VALIDATION

Test Case	Scenario	Result
TC1	Correct guess	PASS
TC2	Incorrect guess	PASS
TC3	Duplicate guess prevention	PASS
TC4	Invalid input handling	PASS
TC5	Win condition detection	PASS
TC6	Loss condition detection	PASS
TC7	Multiple rounds	PASS
TC8	Game statistics accuracy	PASS

All 8 test cases passed successfully. The game is fully functional and production-ready.

6. FILES DELIVERED

hangman_game.py
Main Python script containing the complete game implementation. Can be executed directly from command line. Approximately 150 lines of well-commented, modular code.

Hangman_Game_CodeAlpha.ipynb
Google Colab-compatible Jupyter Notebook with the complete implementation broken into 10 educational sections. Each function is explained and tested individually before being integrated into the full game.

PROJECT_REPORT.md
Comprehensive markdown report documenting the entire project including overview, requirements, implementation details, features, testing results, user guide, and learning outcomes.

README.md
GitHub-ready documentation with project overview, installation instructions, how-to-play guide, technical details, code quality information, and future enhancement suggestions.

WORD_LIST.txt
Reference file containing the 5 predefined words with descriptions, statistics, and information on how to expand the word list.

CodeAlpha_HangmanGame_Report.pdf
This professional PDF report providing a formatted, printable version of the complete project documentation.

7. LEARNING OUTCOMES

Python Fundamentals Practiced:
• Variables and data types (strings, lists, integers)
• Control flow (if-else statements, while loops)
• Functions and parameters
• List operations and string manipulation
• Input/output operations
• Error handling and input validation

Programming Best Practices Demonstrated:
• Code organization using functions
• Clear and descriptive naming conventions
• Comprehensive comments and documentation
• Input validation and error handling
• User-friendly interface design
• Game state management
• Modular and reusable code architecture

8. HOW TO RUN THE PROJECT

Option 1: Direct Python Execution
Open command prompt and navigate to the project directory, then execute:
`python hangman_game.py`

Option 2: Jupyter Notebook (Local)
Install Jupyter if not already installed, then run:
`jupyter notebook Hangman_Game_CodeAlpha.ipynb`

Option 3: Google Colab (Cloud)
1. Visit `https://colab.research.google.com`
2. Upload the `Hangman_Game_CodeAlpha.ipynb` file
3. Run all cells in sequence
4. Play the game in the browser

Requirements:
• Python 3.6 or higher
• No external libraries required (only built-in modules used)

9. PROJECT STATISTICS

Metric	Value
Total Lines of Code	~150
Number of Functions	7
Number of Words	5
Max Incorrect Guesses	6
Hangman Stages	7
Time to Implementation	~2 hours
Test Cases	8
Test Pass Rate	100%

10. NEXT STEPS FOR SUBMISSION

Step 1: Create GitHub Repository
Create a new repository named `CodeAlpha_HangmanGame` and upload all project files.

Step 2: Create LinkedIn Post
Share the project on LinkedIn with @CodeAlpha tag, including a description and link to the GitHub repository.

Step 3: Record Video Explanation
Create a 5-10 minute video demonstrating the game functionality and explaining the code. Upload to

YouTube or share as a file.

Step 4: Submit via CodeAlpha Form
Complete and submit the official CodeAlpha submission form with links to GitHub, LinkedIn post, and video explanation.

Step 5: Complete Optional Task (Task 2)
For additional portfolio strengthening, consider completing Task 2 (Stock Portfolio Tracker) as well.

Project Status:	**COMPLETE**
Submission Ready:	**YES**
Certificate Eligible:	**YES**
Report Generated:	**May 31, 2026 at 00:23:27**

CodeAlpha Contact Information:
Website: www.codealpha.tech | WhatsApp: +91 8052293611 | Email: services@codealpha.tech

Project Author:
Abdul Samad | BS Mathematics, Sukkur IBA University | CodeAlpha Python Programming Internship

This project is submitted as part of the CodeAlpha Python Programming Internship program.